



Artículo

Arquitectura de codiseño hardware-software para EMG en tiempo real

Procesamiento de características en sistemas protésicos basados en FPGA

Carlos Gabriel Mireles Preciado ¹ , Diana Carolina Toledo-Pérez ² , Roberto Augusto Gómez-Loenzo ^{2,*} ,
Marcos Avilés ¹ y Juvenal Rodríguez-Reséndiz ^{2,*}

¹ Facultad de Informática, Universidad Autónoma de Querétaro, Querétaro 76230, México; gsobrinio56@gmail.com (CGM-P.); marcosaviles@ieee.org (MA)

² Facultad de Ingeniería, Universidad Autónoma de Querétaro, Querétaro 76010, México; dtoledo16@alumnos.uaq.mx

* Correspondencia: rob@uaq.mx (RAG-L.); juvenal@uaq.edu.mx (JR-R.)

Abstracto

Este artículo presenta una novedosa arquitectura de hardware para implementar la extracción de características EMG en tiempo real y la reducción de dimensionalidad en entornos FPGA con recursos limitados. La arquitectura de coprocesamiento propuesta integra cuatro extractores de características en el dominio del tiempo (MAV, WL, SSC, ZC) con una unidad especializada de multiplicación de matrices PCA dentro de una tubería de procesamiento unificada, lo que demuestra mejoras significativas en la eficiencia energética y la latencia de procesamiento en comparación con los enfoques tradicionales basados en software. Se evalúan múltiples arquitecturas de multiplicación de matrices para optimizar la utilización de los recursos de la FPGA, manteniendo un rendimiento determinista en tiempo real, utilizando una placa de evaluación Zed como plataforma de desarrollo. Esta implementación logra una reducción de dimensionalidad eficiente con recursos de hardware mínimos, lo que la hace idónea para aplicaciones de prótesis. La funcionalidad de este sistema se valida mediante una base de datos EMG personalizada de estudios previos. Los resultados demuestran una mejora de velocidad de 7,3 veces y una ganancia de eficiencia energética de 3,1 veces en comparación con la implementación de software ARM Cortex-A9, lo que valida el enfoque arquitectónico para aplicaciones de control de prótesis alimentadas por batería.

Palabras clave: FPGA; análisis de componentes principales; electromiografía; extracción de características; procesamiento en tiempo real; codiseño hardware-software



Editora académica: Maryam Ravan

Recibido: 21 de agosto de 2025

Revisado: 22 de septiembre de 2025

Aceptado: 27 de septiembre de 2025

Publicado: 30 de septiembre de 2025

Cita: Mireles-Preciado, CG;

Toledo-Pérez, DC; Gómez-Loenzo,

REAL ACADEMIA DE BELLAS ARTES; Avilés, M.;

Rodríguez-Reséndiz, J. Codiseño hardware-software

Arquitectura para EMG en tiempo real

Procesamiento de características en sistemas
protésicos basados en FPGA. Algorithms 2025,

18, 617. <https://doi.org/10.3390/a18100617>

a18100617

Derechos de autor: © 2025 por los autores.

Licenciatario MDPI, Basilea, Suiza.

Este artículo es de acceso abierto.

distribuido bajo los términos y

condiciones de Creative Commons

Licencia de atribución (CC BY)

(<https://creativecommons.org/licenses/by/4.0/>).

1. Introducción

Los arreglos de puertas programables en campo (FPGA) se han consolidado como plataformas potentes para la implementación de algoritmos complejos de procesamiento de señales digitales, ofreciendo ventajas únicas en capacidades de procesamiento paralelo y rendimiento en tiempo real, a la vez que mantienen un bajo consumo de energía [1]. Esta combinación de características ha hecho que los FPGA sean cada vez más atractivos para aplicaciones biomédicas portátiles, donde la eficiencia energética y la velocidad de procesamiento impactan directamente en la usabilidad del dispositivo y los resultados para el paciente [2]. La reconfigurabilidad de los FPGA también permite arquitecturas de procesamiento adaptativas optimizadas para requisitos específicos de procesamiento de señales biomédicas [3].

En aplicaciones biomédicas, las FPGA han demostrado éxito en diversos dominios, incluyendo el análisis de ECG [4,5], las interfaces cerebro-computadora [6] y el procesamiento de bioseñales en tiempo real [7]. La electromiografía (EMG) mide la actividad eléctrica producida por los músculos esqueléticos durante la contracción y la relajación [8,9]. Estas señales son inherentemente complejas, contienen información útil sobre la actividad muscular pero, desafortunadamente, también varios tipos

del ruido de fuentes fisiológicas y ambientales [10]. La naturaleza de alta dimensión de las señales EMG sin procesar, combinada con sus características no estacionarias, presenta desafíos significativos para el procesamiento y análisis en tiempo real [11,12].

Si bien los sistemas de procesamiento de EMG existentes suelen implementar la extracción de características y la reducción de dimensionalidad como módulos de software separados que se ejecutan en procesadores de propósito general, este enfoque presenta problemas significativos de latencia y consumo de energía en aplicaciones portátiles. Estudios recientes [13] han demostrado que el análisis de componentes principales (PCA) y el coprocesamiento en FPGA para bioseñales, en particular EMG, están ganando terreno. La innovación arquitectónica clave que se presenta en este trabajo es la cointegración de la extracción de características en el dominio del tiempo con la transformación PCA en una única canalización de procesamiento en FPGA, lo que elimina los cuellos de botella en la transferencia de datos y permite un procesamiento determinista en tiempo real adecuado para aplicaciones de control de prótesis.

Esto exige arquitecturas de procesamiento eficientes que operen dentro de las limitaciones de los dispositivos portátiles; su implementación para el procesamiento de señales EMG presenta desafíos únicos, particularmente en sistemas de control de prótesis [14,15]. Los enfoques tradicionales basados en software a menudo tienen dificultades para cumplir con estos requisitos, especialmente en aplicaciones en tiempo real donde la latencia de procesamiento impacta directamente la experiencia del usuario [16].

Si bien se han explorado implementaciones de PCA en FPGA para diversas aplicaciones, el procesamiento de señales EMG para el control de prótesis presenta desafíos únicos que las implementaciones de PCA de propósito general no abordan adecuadamente. A diferencia de muchas otras aplicaciones, el control de prótesis basado en EMG requiere: (1) un funcionamiento de ultrabajo consumo para maximizar la duración de la batería en contextos portátiles, (2) un procesamiento determinista en tiempo real con latencias inferiores a 300 ms para mantener la percepción natural del movimiento, y (3) una precisión suficiente con una utilización mínima de recursos para permitir la integración en dispositivos protésicos compactos. Estas restricciones específicas requieren enfoques arquitectónicos especializados que van más allá de las implementaciones estándar de multiplicación de matrices.

La precisión y fiabilidad de estas aplicaciones dependen en gran medida de técnicas eficaces de procesamiento de señales [17]. El PCA se ha consolidado como una potente herramienta para el procesamiento de señales EMG, ofreciendo múltiples ventajas tanto para el análisis como para las aplicaciones prácticas [18], como por ejemplo, un reconocimiento de patrones más eficiente en aplicaciones protésicas [19]. La capacidad de la técnica para reducir la dimensionalidad de los datos conservando las características esenciales de la señal la hace especialmente valiosa para sistemas de procesamiento en tiempo real [20,21]. Al aplicar PCA a las señales EMG, los investigadores eliminan el ruido y los artefactos que interfieren con el control de las prótesis [22,23], a la vez que extraen características relevantes para el reconocimiento de patrones [24,25], lo que permite una identificación más precisa de los movimientos y gestos intencionados.

Sin embargo, la implementación de PCA en FPGA requiere una consideración cuidadosa de la utilización de recursos y la arquitectura de procesamiento para mantener el rendimiento en tiempo real y minimizar el consumo de energía [26].

Las recientes implementaciones de PCA en FPGA para aplicaciones biomédicas han explorado diversos enfoques arquitectónicos. Entre las contribuciones recientes más relevantes se incluyen aceleradores de PCA basados en FPGA [27], preprocesamiento de EMG de bajo consumo energético [28,29] y soluciones de coprocesadores [30,31]. Algunas implementaciones se han centrado en maximizar la precisión mediante aritmética de punto flotante, mientras que otras han priorizado la eficiencia de los recursos mediante aritmética de punto fijo y operaciones matriciales optimizadas [32]. Sin embargo, estos enfoques a menudo no logran un equilibrio óptimo entre la utilización de recursos y la precisión del procesamiento, especialmente para los requisitos específicos del procesamiento de EMG en aplicaciones protésicas [33].

La creciente complejidad de los sistemas de control de prótesis ha enfatizado aún más la necesidad de técnicas eficientes de reducción de dimensionalidad. Los dispositivos protésicos modernos a menudo incorporan múltiples canales EMG y algoritmos de control sofisticados, lo que hace que la optimización de los pasos de preprocesamiento, incluida la extracción de características y la reducción de dimensionalidad,

cada vez más crítico [34]. Las restricciones en tiempo real de las aplicaciones de control de prótesis, que normalmente requieren tiempos de respuesta inferiores a 300 ms para el movimiento natural, añaden otra capa de complejidad para el desafío de implementación [17].

Este artículo presenta un estudio de caso arquitectónico que demuestra las ventajas de Integración de la extracción de características EMG con la reducción de dimensionalidad en FPGA embebida sistemas. Las principales contribuciones son las siguientes:

- Una novedosa arquitectura de coprocesamiento que integra la extracción de características en el dominio del tiempo. (Valor absoluto medio, VAM; Longitud de la forma de onda, LW; Cambios de signo de la pendiente, CSP; Cero Cruces, ZC) con transformación PCA en una tubería FPGA unificada, eliminando cuellos de botella en el procesamiento de software tradicional.
- Análisis arquitectónico comparativo de tres implementaciones de multiplicación de matrices (completo, basado en bloques y basado en columnas), demostrando una utilización óptima de los recursos. Estrategias para el procesamiento de EMG en entornos con recursos limitados.
- Evaluación cuantitativa del rendimiento que muestra una mejora de velocidad de 7,3× y una ganancia de eficiencia energética de 3,1× en comparación con la implementación de software ARM Cortex-A9, lo que valida El enfoque arquitectónico para aplicaciones de prótesis alimentadas por batería.
- Una metodología completa de codiseño de hardware y software que utiliza la plataforma Zynq SoC que Permite la creación rápida de prototipos y la validación de algoritmos de procesamiento de EMG en sistemas embebidos.

El sistema propuesto aprovecha las capacidades de procesamiento de la plataforma Zynq SoC , combinando la flexibilidad de un sistema operativo basado en Linux con el rendimiento. ventajas de los aceleradores de hardware personalizados. Este enfoque híbrido permite una implementación eficiente tanto de las etapas de extracción de características como de las de cálculo de PCA, manteniendo al mismo tiempo flexibilidad del sistema para futuras actualizaciones y modificaciones [35].

El resto de este artículo está organizado de la siguiente manera. La sección 2 describe la teoría Antecedentes de PCA y extracción de características EMG, incluyendo los fundamentos matemáticos y consideraciones de implementación. La Sección 3 presenta una descripción detallada de La estructura de la base de datos y la implementación del sistema experimental, centrándose en la arquitectura del hardware y la canalización de procesamiento. La sección 4 analiza los resultados experimentales. y análisis de rendimiento, incluyendo métricas de utilización de recursos y latencia de procesamiento. mediciones. Finalmente, la Sección 5 concluye el artículo y sugiere direcciones para el futuro. Investigación en procesamiento de señales basado en FPGA para aplicaciones protésicas.

2. Métodos

2.1. Métricas de extracción de características EMG implementadas por hardware

En el Apéndice A se describen varias características en diferentes dominios. En esta sección, describimos las cuatro características implementadas en hardware (HW) como parte de la FPGA. firmware (FW), que se resumen en la Tabla 1.

Tabla 1. Resumen de las cuatro métricas implementadas en el sistema.

Propósito principal del sistema métrico		Sensibilidad	Aplicaciones
MAV mide la amplitud promedio.		Amplitud de la señal.	Activación muscular, fatiga.
WL	Captura la complejidad de la señal y energía.	Amplitud y frecuencia. Extracción de características, análisis de señales.	
SSC	Mide el contenido de frecuencia.	Cambios en la pendiente.	Detección de movimiento, fatiga.
ZC mide la frecuencia de la señal.		Eventos de cruce por cero.	Reconocimiento de patrones, control de movimiento.

2.1.1. VMA: Valor absoluto medio

Definición: MAV es el promedio de los valores absolutos de la señal EMG durante un cierta ventana o segmento de N muestras:

$$MAV = \frac{1}{N} \sum_{i=1}^N |x_i|,$$

donde x_i es el valor de la señal EMG en la muestra i , y N es el número de muestras en el segmento.

Aplicación: MAV refleja el nivel de activación muscular y se utiliza ampliamente en el entrenamiento muscular. Estudios de fatiga y control de prótesis.

2.1.2. WL—Longitud de la forma de onda

Definición: WL es la longitud acumulada de la forma de onda durante un segmento de tiempo específico:

$$WL = \sum_{i=1}^{N-1} |x_{i+1} - x_i|,$$

donde x_i es el valor de la señal EMG en la muestra i , y N es el número de muestras en el segmento. Representa la complejidad y la energía de la señal.

Aplicación: WL es sensible tanto a los cambios de amplitud como de frecuencia en la señal EMG, lo que la hace útil para la extracción de características en tareas de clasificación.

2.1.3. SSC—Cambios de signo de pendiente

Definición: SSC cuenta el número de veces que cambia la pendiente de la señal EMG. dirección, superando un umbral predefinido:

$$SSC = \sum_{i=2}^{N-1} (x_i - x_{i-1}) \cdot (x_{i+1} - x_i) < 0 \quad |x_{i+1} - x_i| > \Delta,$$

donde Δ es el valor umbral; mide el contenido de frecuencia de la señal.

Aplicación: Útil para identificar características de señales como contracciones musculares o fatiga.

2.1.4. ZC—Cruces por cero

Definición: ZC cuenta el número de veces que la señal EMG cruza la amplitud cero. línea dentro de un segmento, considerando un umbral para la reducción de ruido:

$$ZC = \sum_{i=1}^{N-1} (x_i \cdot x_{i+1} < 0) \quad |x_i - x_{i+1}| > \Delta,$$

donde Δ es el valor umbral.

Aplicación: ZC es un indicador de frecuencia de señal y se utiliza a menudo en patrones. reconocimiento para la clasificación del movimiento.

2.2. Arquitectura de cálculo de matrices PCA eficiente en FPGA

Dado un conjunto de n muestras (escaladas adecuadamente si es necesario) $x_1, x_2, \dots, x_n \in \mathbb{R}^m$, forman el conjunto de datos.

$$X = [x_1 \ x_2 \ \dots \ x_n].$$

Cada una de las m características $x_{1,j}, x_{2,j}, \dots, x_{m,j}$ de una muestra particular x_j proporciona una cantidad diferente de "información" que generalmente está correlacionada hasta cierto grado con las otras características. El objetivo principal del Análisis de Componentes Principales (PCA) es "desenredar" esta información, es decir, detectar los datos más relevantes, que se asignarán a un modelo de datos r -dimensional.

subespacio con $r \leq m$. En otras palabras, el objetivo es reducir la dimensionalidad de cada muestra x_j reemplazándola con un vector x de r dimensiones (con $r \leq m$) que contiene casi la misma cantidad de información pero, con una dimensión reducida, se procesa más rápido con la misma potencia de cálculo. Este nuevo conjunto de datos

$$X' = [x_1' \ x_2' \ \dots \ x_n']^T, \quad r \leq m,$$

se calcula de la siguiente

manera: • Se calcula el promedio/media por fila,

$$\mu_X = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_m \end{bmatrix} = \begin{bmatrix} \frac{1}{n} \sum_{j=1}^n x_{1,j} \\ \frac{1}{n} \sum_{j=1}^n x_{2,j} \\ \vdots \\ \frac{1}{n} \sum_{j=1}^n x_{m,j} \end{bmatrix}$$

• Luego, se resta de cada muestra en el conjunto de datos,

$$X - \mu_X \mathbf{1} = \begin{bmatrix} x_{1,1} - \mu_1 & x_{1,2} - \mu_1 & \dots & x_{1,n} - \mu_1 \\ x_{2,1} - \mu_2 & x_{2,2} - \mu_2 & \dots & x_{2,n} - \mu_2 \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} - \mu_m & x_{m,2} - \mu_m & \dots & x_{m,n} - \mu_m \end{bmatrix}$$

(recuerde que $u^T v = uv^T$), donde $\mathbf{1} = (1, \dots, 1)^T \in \mathbb{R}^n$.
Para calcular la matriz de covarianza,

$$\Sigma = \frac{1}{n-1} (X - \mu_X \mathbf{1}) (X - \mu_X \mathbf{1})^T \in \mathbb{R}^{m \times m}.$$

• Dado que Σ es una matriz simétrica, sus vectores propios $v_1, \dots, v_m \in \mathbb{R}^m$ y sus correspondientes valores propios $\lambda_1, \dots, \lambda_m$ se calculan utilizando el algoritmo de valores propios de Jacobi [36] para obtener

$$\Lambda = V^{-1} \Sigma V,$$

dónde

$$V = [v_1 \ v_2 \ \dots \ v_m],$$

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_m \end{bmatrix};$$

observe que, dado que Σ es simétrica con entradas reales, V es ortogonal, es decir, $V^{-1} = V^T$.

Para reducir los datos a una dimensión r , se toman los r autovectores $v_1, \dots, v_r$ asociados con los r autovalores más grandes $\lambda_1, \dots, \lambda_r$ (los primeros r autovalores son los autovalores más grandes, es decir,

$$|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_r|,$$

dado que los valores propios y sus vectores propios correspondientes se reordenan para garantizar esto), y luego los datos originales se proyectan sobre el subespacio vectorial generado por $v_1, \dots, v_r$, es decir, las coordenadas $t_{1,j}, t_{2,j}, \dots, t_{r,j}$ se determinan de tal manera que

$$\text{proj } x_j = t_{1,j}v_1 + t_{2,j}v_2 + \dots + t_{r,j}v_r.$$

Nótese que, por definición, cada $x_j - \text{proj } x_j$ debe ser ortogonal a cada vector en el espacio generado por $v_1, v_2, \dots, v_r$, es decir

$$\begin{aligned} (x_j - \text{proj } x_j) \cdot v_1 &= 0, \\ (x_j - \text{proj } x_j) \cdot v_2 &= 0, \\ &\vdots \\ (x_j - \text{proj } x_j) \cdot v_r &= 0, \end{aligned}$$

lo cual es equivalente a,

$$\begin{aligned} 1 \quad & [x_j - (t_{1,j}v_1 + t_{2,j}v_2 + \dots + t_{r,j}v_r)] = 0, \\ 2 \quad & [x_j - (t_{1,j}v_1 + t_{2,j}v_2 + \dots + t_{r,j}v_r)] = 0, \\ & \vdots \\ r \quad & [x_j - (t_{1,j}v_1 + t_{2,j}v_2 + \dots + t_{r,j}v_r)] = 0. \end{aligned}$$

Al usar las notaciones $V_r = v_1 v_2 \dots v_r$ y $t_j = (t_{1,j}, t_{2,j}, \dots, t_{r,j})$, se reescribe diez como

$$\begin{aligned} 1 \quad & [x_j - V_r t_j] = 0, \\ 2 \quad & [x_j - V_r t_j] = 0, \\ & \vdots \\ r \quad & [x_j - V_r t_j] = 0, \end{aligned}$$

eso es,

$$V_r^T (x_j - V_r t_j) = 0,$$

y por lo tanto

$$V_r^T V_r t_j = V_r^T x_j.$$

Dado que los vectores $v_1, v_2, \dots, v_r$ son ortogonales entre sí (porque son vectores propios de la matriz simétrica Σ), $V_r^T V_r = I$, lo que simplifica la reducción de dimensionalidad a

$$t_j = V_r^T x_j.$$

Si el conjunto de datos X se transpone, como suele ocurrir cuando las muestras se almacenan como filas y características como columnas, debemos calcular

$$t_j^T = x_j^T V_r^T, \quad (1)$$

o, para todo el conjunto de datos,

$$T^T = X^T V_r.$$

El cálculo basado en FPGA de (1) se describe en detalle en la Sección 3.2.3. Dado que x_j^T se calcula, como se describe en la Sección 2.1, en la misma FPGA, no hay necesidad de

transferir los datos del microprocesador una vez que V_r se almacena en la FPGA, lo que acelera enormemente el rendimiento (véase la figura 10).

2.3. Fundamentación del diseño arquitectónico

La selección de características del dominio del tiempo (MAV, WL, SSC, ZC) y PCA para este estudio arquitectónico se basa en la compatibilidad computacional más que en la novedad algorítmica. Estas características se eligieron porque:

1. Sencillez del hardware: Solo requieren operaciones aritméticas básicas (suma, resta, comparación) que se asignan de manera eficiente a los recursos de la FPGA.
2. Compatibilidad con la canalización: El patrón de computación secuencial permite una transmisión eficiente. arquitectura con requisitos mínimos de memoria.
3. Base de referencia establecida: Los algoritmos bien documentados permiten centrarse en la optimización arquitectónica en lugar del desarrollo algorítmico.

La principal contribución de esta investigación no reside en la selección de estos métodos, sino en la demostración de que la integración conjunta de la extracción de características con la reducción de dimensionalidad en una única arquitectura FPGA proporciona ventajas de rendimiento significativas con respecto a los enfoques de procesamiento separados tradicionales.

3. Base de datos y sistema experimental En las siguientes secciones,

describiremos la base de datos utilizada en la plataforma de experimentación y proporcionaremos una descripción detallada del desarrollo de hardware (HW) y software (SW) realizado para este artículo.

3.1. Base de datos

La base de datos comprende datos EMG recopilados de ocho sujetos sanos (cuatro mujeres y cuatro hombres) mediante cuatro sensores (S1, S2, S3 y S4) ubicados en puntos específicos de la región transtibial, el área entre la rodilla y el tobillo donde normalmente se colocaría una prótesis por debajo de la rodilla. Dado que los sistemas de control de prótesis requieren una calibración individual para cada usuario, la base de datos prioriza múltiples repeticiones de movimiento por sujeto (20 repeticiones $\times$ 6 movimientos) en lugar de un gran número de sujetos. Este enfoque se ajusta a la naturaleza personalizada del control de prótesis, donde el rendimiento del sistema depende de la adaptación a los patrones EMG individuales en lugar de la generalización a toda la población.

Para la recopilación de datos, se pidió a cada participante que realizara 20 repeticiones de 6 movimientos distintos (AP, AT, LP, LT, PD y PI), además de un estado de referencia de reposo (sin movimiento ni contracción). En cada repetición, se registraron los datos de los cuatro sensores con una frecuencia de muestreo de 1 ms, y se almacenó una ventana de 5150 muestras en la base de datos, lo que suma un total de 103 000 muestras por movimiento.

Los diferentes movimientos que se muestrean se describen a continuación: AP Apoyar la punta del pie sin levantar el talón.

AT Sostenga el talón sin levantar la punta del pie.

LP Levanta el dedo del pie.

LT Levanta el talón.

PD Mueva la punta hacia la derecha.

PI Mueva la punta hacia la izquierda.

RR Estado de relajación o descanso.

En la columna izquierda de la Figura 1 se muestra una sola repetición del movimiento AP. Se calcula la media de los datos y se resta de los datos originales para obtener un conjunto de datos con media cero, como se muestra en la segunda columna de la misma figura. El siguiente paso consiste en identificar el punto de inicio del movimiento, que se muestra en la tercera columna.

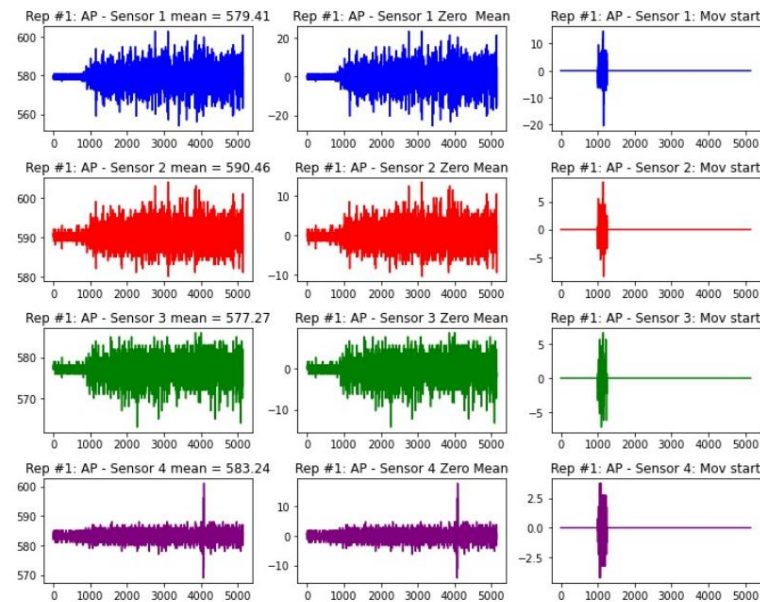


Figura 1. Repetición del movimiento AP. El eje horizontal está escalado en muestras, mientras que el eje vertical está escalado en recuentos ADC. La primera columna muestra la señal original, la segunda columna es la señal menos su media, y la tercera columna muestra solo las primeras 250 muestras después de la inicio del movimiento.

También es importante observar cómo se distribuyen estadísticamente los datos. Esto nos permite observe la naturaleza de los datos. En la Figura 2 se muestra una vista de la distribución estadística .

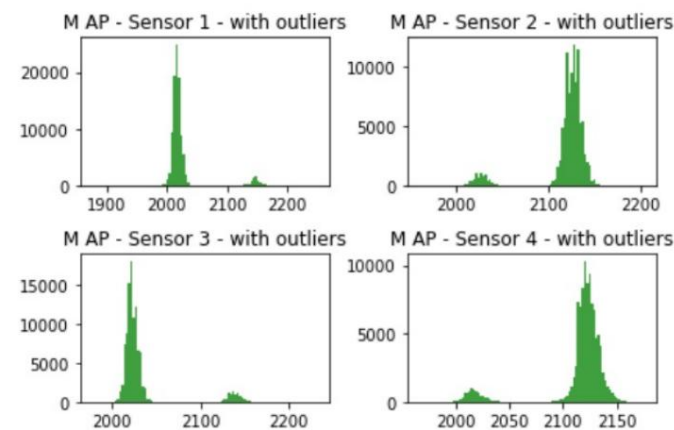


Figura 2. Distribución estadística (histograma) del movimiento del AP con los datos brutos de los sensores. El eje horizontal agrupa el conteo del ADC antes del escalado, y el eje vertical agrupa la distribución de frecuencia.

3.2. Plataforma de experimentación

La plataforma de experimentación se basa en una placa Zed Board de Digilent™ que incorpora un dispositivo Xilinx™ Zynq XC7Z020CLG484-1 (Xilinx Inc. 2100 Logic Drive, San José, CA, EE. UU.).

3.2.1. Justificación de la elección de diseño

La selección de una implementación de lógica programable (PL) para las operaciones de multiplicación de matrices en el sistema de procesamiento (PS) se basó en consideraciones de rendimiento cuantitativo. Si bien el ARM Cortex-A9 del PS ofrece unidades de punto flotante dedicadas, nuestras pruebas preliminares revelaron que un enfoque acelerado por hardware mediante PL proporcionó un rendimiento superior para esta aplicación específica, como se muestra en la Tabla 2.

Tabla 2. Comparación de la implementación de PL frente a PS para operaciones de PCA.

Implementación	Tiempo de procesamiento (µs)	Potencia (mW)
PS (ARM Cortex-A9)	660	1531
PL (Lógica personalizada)	90	489

La implementación de PL logra un procesamiento aproximadamente 7,3 veces más rápido con un consumo de energía 3,1 veces menor , lo que resulta en una mejora de 22,63 veces en la eficiencia energética por operación. Esto es fundamental para las aplicaciones protésicas, donde la duración de la batería influye directamente en la facilidad de uso. Además, la naturaleza determinista de la implementación de PL proporciona garantías de latencia consistentes, esenciales para el control de prótesis en tiempo real, a diferencia de PS, que puede experimentar una latencia variable debido a la sobrecarga del sistema operativo.

En cuanto a nuestra elección de IP de punto flotante, evaluamos varias opciones, incluyendo los núcleos IP de Xilinx proporcionados por el proveedor y la biblioteca de la Universidad Johns Hopkins. Seleccionamos la biblioteca de punto flotante de JHU principalmente por su implementación ligera, portabilidad y configurabilidad, lo que nos permitió optimizar nuestros requisitos específicos de procesamiento EMG. Si bien la biblioteca data de 2011, logramos acelerar su operación de frecuencia de 25 MHz a 100 MHz; nuestras pruebas de rendimiento mostraron que, para sumas y restas de precisión simple en nuestro caso de uso particular, proporcionó un 12,5 % menos de utilización de recursos en comparación con la IP equivalente de Xilinx, con solo un aumento del 7 % en la latencia. Mitigamos los problemas de confiabilidad utilizando IP del proveedor para operaciones de división, donde la biblioteca de JHU mostró resultados inconsistentes en nuestras pruebas.

3.2.2. Descripción del hardware

El sistema descrito anteriormente utiliza un sistema en chip (SoC) FPGA. El FPGA contiene un sistema de procesamiento (PS) basado en un procesador ARM A9 de doble núcleo y un área de lógica programable (PL) que se comunican entre sí mediante la interfaz AXI4-Lite. La figura 3 muestra cómo se comunican la sección de lógica programable y la sección de procesamiento en el sistema FPGA-SoC.

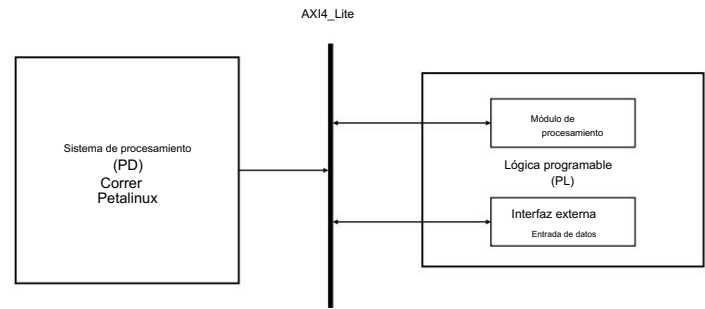


Figura 3. Uso de la interfaz AXI4-Lite para la comunicación entre los lados PS y PL. La comunicación con los módulos implementados en la sección PL se realiza a través de la interfaz AXI4-Lite.

La implementación de las cuatro métricas se logra mediante la creación de máquinas de estados que reciben una señal (ver dataValid en la Sección 4.2) y luego procesan los datos de entrada; este módulo "Métricas" calcula las cuatro métricas descritas en la Sección 2.1 para cada sensor, produciendo una salida de dieciséis parámetros que, después de este proceso, se entregan al módulo "PCA" que procesa el Análisis de Componentes Principales.

Los datos de entrada se envían a los módulos de procesamiento mediante una interfaz externa desarrollada con un programa en lenguaje C o mediante la lectura de la memoria BRAM dentro del FPGA. Estos métodos se utilizan para comprobar el procesamiento y verificar los resultados. Se integró una interfaz UART en la sección PL para agilizar el proceso de entrada de datos para fines de simulación. Aunque en la sección PS hay interfaces seriales disponibles, era mejor usar una interfaz inter-

La interfaz en la PL se encarga de entregar los datos entrantes directamente a los módulos implementados para los algoritmos que se ejecutan en la PL. De esta manera, se evitan retrasos y accesos innecesarios a la capa de Linux; por lo tanto, los procesos de simulación se completan más rápidamente. La figura 4 muestra el diagrama de bloques desde una vista superior.

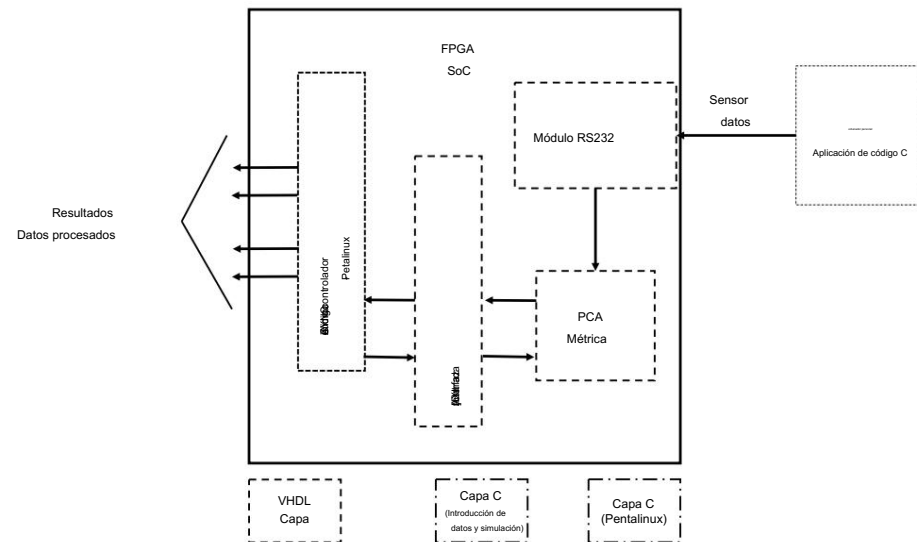


Figura 4. Diagrama de bloques del sistema que muestra la interfaz externa de entrada de datos y la interconexión entre las secciones PS y PL. Las diferentes capas están codificadas por colores.

El sistema se implementó utilizando Vivado 2019.1 y Petalinux 2019.1 (para garantizar la compatibilidad entre los componentes de hardware y software), como se muestra en la Figura 4.

La arquitectura de hardware descrita usando Vivado se ilustra en el diagrama de bloques de la Figura 5, que consta de varios módulos clave: • El

módulo AXI I/F es la interfaz de comunicación principal entre el PL y el PS; esta interfaz

implementa un protocolo de comunicación basado en registros que permite la transferencia de datos entre el PS, donde reside la pila de software Petalinux, y dos módulos de procesamiento primarios en el PL: el módulo "Metrics" y el módulo "PCA".

- El bloque principal desde la perspectiva de dónde se realizan los cálculos del algoritmo es el módulo "Procesamiento" que se muestra como RTL cerca del centro de la Figura 5.

Este módulo coordina la entrada de datos de los sensores y los envía al módulo de "Métricas".

- El módulo "Métricas" está encapsulado dentro del módulo "Procesamiento" y calcula las cuatro métricas para cada sensor descrito en la Sección 2.1, produciendo así 16 parámetros que se convierten en los datos de entrada x_j para el módulo "PCA".
- El módulo "PCA" también se almacena en el bloque del módulo RTL y realiza el cambio de coordenadas de x_j a t_j dado en la Ecuación (1) utilizando el procedimiento descrito en la Sección 3.2.3. La salida t_j del módulo PCA se recoge para revisar los primeros r componentes.

Estos dos últimos módulos dependen de los módulos representados a la izquierda, que son

multiplicadores y divisores. Los divisores en el módulo "Métricas" realizan las operaciones de división necesarias principalmente dentro de la métrica MAV. Los multiplicadores sirven para calcular el producto de las métricas calculadas x_j con la matriz PCA V (que se calcula previamente en el programa en lenguaje C que se ejecuta en SW en Petalinux) como en la Ecuación

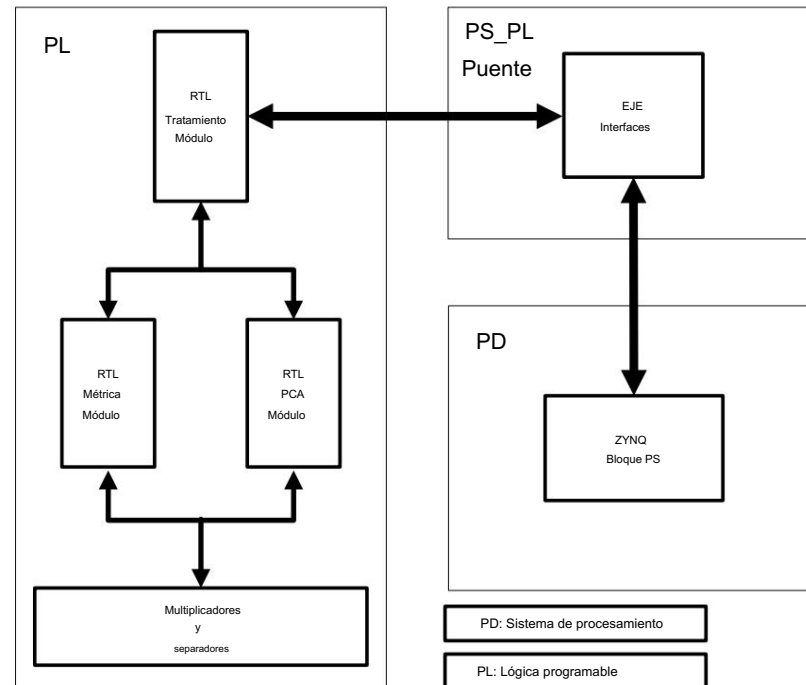


Figura 5. Diagrama de bloques del sistema que muestra la interfaz de entrada externa.

Este trabajo utiliza una biblioteca VHDL desarrollada en la Universidad Johns Hopkins para operaciones de punto flotante [37]. La documentación de esta biblioteca indica que se probó principalmente para operaciones de suma, resta y producto, lo que refuerza la observación de que el módulo de división podría requerir atención. De hecho, en este trabajo, los módulos de multiplicación y división a veces, durante las simulaciones y en el hardware real, arrojaron resultados que presentaban discrepancias con los valores esperados, y es por esta razón que se decidió utilizar el núcleo IP de punto flotante externo de Xilinx, que es capaz de realizar varias operaciones, entre ellas las de multiplicación y división. El equipo de Johns Hopkins declara que probaron sus módulos a 25 MHz, y que en esta aplicación se requiere que el complemento utilizado funcione a 100 MHz, y que funciona perfectamente, sin que se hayan detectado errores.

Todos los módulos PL están implementados mediante máquinas de estados finitos en VHDL. La figura 6 muestra la máquina de estados del módulo “PCA”.

La figura 7 muestra la comunicación entre los tres módulos principales (“Procesamiento”, “Métricas” y “PCA”) se describen en VHDL. El módulo “Procesamiento” coordina los mensajes entre los tres módulos. El módulo “Métricas” coordina el inicio de los cálculos de las métricas. Una vez completadas las métricas, el módulo superior recibe una interrupción que permite al módulo PCA recibirlas. Las métricas se reciben una vez que se completa el tamaño de ventana N. El parámetro N en este contexto es el mismo que en la Sección 2.1, que define el número de términos en las diferentes métricas descritas allí.

El módulo PCA al recibir las métricas realiza una de dos tareas:

- Enviar el vector de métricas recibidas a una matriz que posteriormente se utilizará para calcular el Matriz PCA, si está en modo de cálculo matricial;
- Procesar el vector de métricas recibidas a través de la matriz PCA, si en operación continua modo ción.

El sistema procesa los vectores de métricas cada vez que están disponibles después de N muestras.

La salida del módulo PCA se transfiere al módulo de procesamiento superior, donde es procesada por el software que se ejecuta en la capa de software superior.

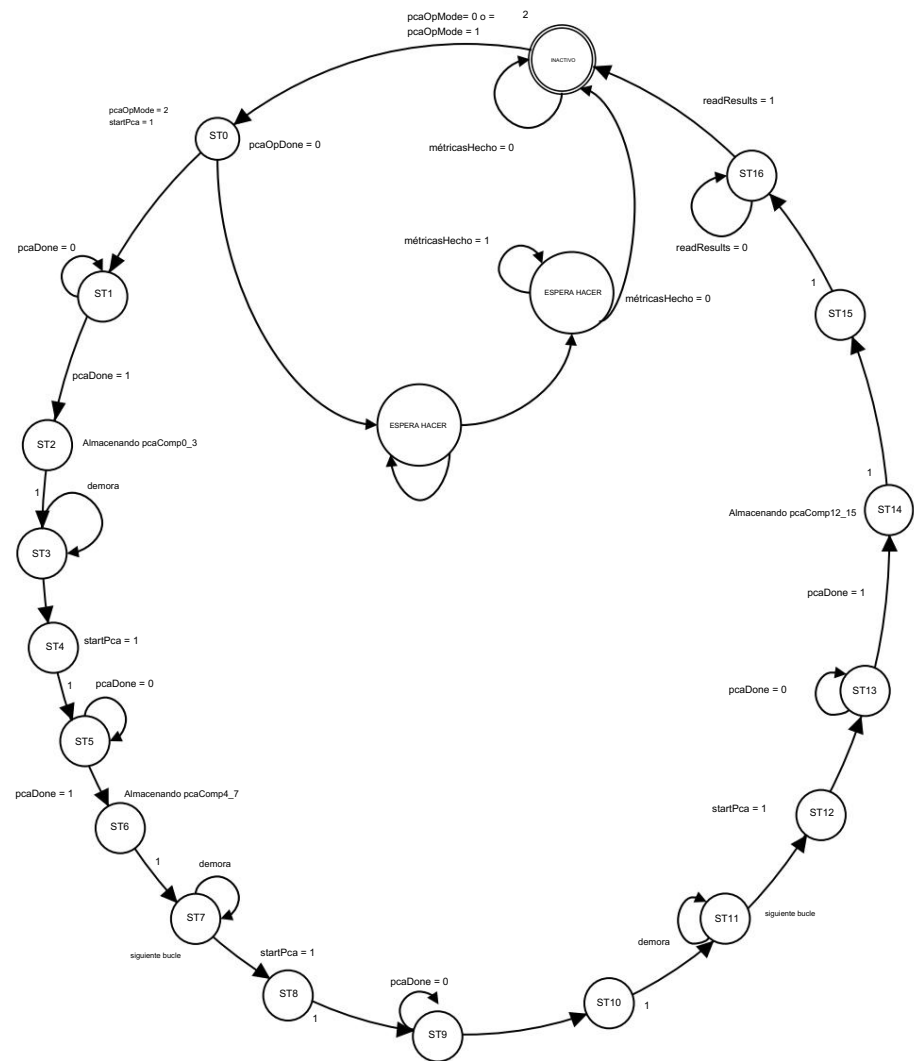


Figura 6. Máquina de estados utilizada en la mayoría de los procesos implementados en VHDL. Este es el caso del proceso “PCA”.

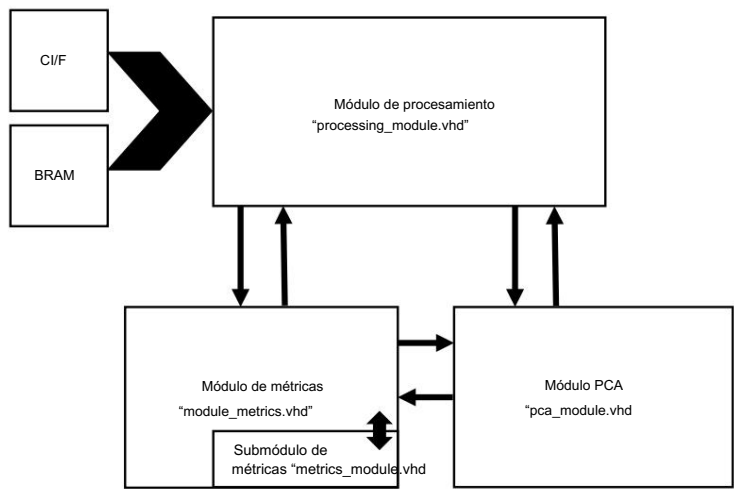


Figura 7. Diagrama que muestra la intercomunicación entre los módulos “PCA”, “Métricas” y el módulo principal “Procesamiento”, que sincroniza los eventos de los otros dos.

3.2.3. Proceso de multiplicación de matrices PCA

El sistema implementado para esta aplicación calcula (1) con $r = m = 16$ multiplicando el vector fila.

$$x_j^T = x_{1,j} x_{2,j} \cdots x_{15,j} x_{16,j}$$

con la matriz

$$V = \begin{matrix} & m_{1,1} & m_{1,2} & \dots & m_{1,15} & m_{1,16} \\ & m_{2,1} & m_{2,2} & \dots & m_{2,15} & m_{2,16} \\ & \vdots & \vdots & & \vdots & \vdots \\ & m_{15,1} & m_{15,2} & \dots & m_{15,15} & m_{15,16} \\ & m_{16,1} & m_{16,2} & \dots & m_{16,15} & m_{16,16} \end{matrix}$$

Nótese que hay cuatro sensores y $x_{4(k-1)+1,j}$, $x_{4(k-1)+2,j}$, $x_{4(k-1)+3,j}$, y $x_{4k,j}$ son las cuatro métricas calculadas a partir del k -ésimo sensor.

La multiplicación vector-matriz es un cuello de botella común en el rendimiento debido a su complejidad computacional y a los recursos que requiere. En este caso, había dos restricciones críticas que debían abordarse:

- La cantidad de recursos lógicos reprogramables disponibles en la FPGA.
- El tiempo de procesamiento que requiere la multiplicación de matrices como parte del algoritmo de procesamiento.

Diferentes arquitecturas para implementar la multiplicación de matrices

$${}^T_j x V = x_{1,j} x_{2,j} \cdots x_{15,j} x_{16,j} \begin{matrix} m_{1,1} & m_{1,2} & \dots & m_{1,15} & m_{1,16} \\ m_{2,1} & m_{2,2} & \dots & m_{2,15} & m_{2,16} \\ \vdots & \vdots & & \vdots & \vdots \\ m_{15,1} & m_{15,2} & \dots & m_{15,15} & m_{15,16} \\ m_{16,1} & m_{16,2} & \dots & m_{16,15} & m_{16,16} \end{matrix} \quad (2)$$

han sido evaluados:

Multiplicación completa: El primer enfoque directo de realizar la multiplicación completa de matrices típica, como se describe en la ecuación (2), fue una consideración temprana. Sin embargo, debido a sus excesivos requisitos de recursos, se consideró inviable para la FPGA objetivo.

Multiplicación de bloques de 4×4 : La segunda opción fue dividir la operación de multiplicación en la ecuación (2) en $4 \times 4 = 16$ operaciones de bloques de la forma

$$\begin{matrix} & a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ & a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ b_0 & b_1 & b_3 & b_4 & \\ & a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \\ & a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{matrix} \quad (3)$$

mediante computación

$$\begin{aligned} & r_{k,4(\ell-1)+1,j} r_{k,4(\ell-1)+2,j} r_{k,4(\ell-1)+3,j} r_{k,4\ell,j} \\ & = x_{4(k-1)+1,j} x_{4(k-1)+2,j} x_{4(k-1)+3,j} x_{4k,j} \\ & \times \begin{matrix} m_{4(k-1)+1,4(\ell-1)+1} & m_{4(k-1)+1,4(\ell-1)+2} & m_{4(k-1)+1,4(\ell-1)+3} & m_{4(k-1)+1,4\ell} \\ m_{4(k-1)+2,4(\ell-1)+1} & m_{4(k-1)+2,4(\ell-1)+2} & m_{4(k-1)+2,4(\ell-1)+3} & m_{4(k-1)+2,4\ell} \\ m_{4(k-1)+3,4(\ell-1)+1} & m_{4(k-1)+3,4(\ell-1)+2} & m_{4(k-1)+3,4(\ell-1)+3} & m_{4(k-1)+3,4\ell} \\ m_{4k,4(\ell-1)+1} & m_{4k,4(\ell-1)+2} & m_{4k,4(\ell-1)+3} & m_{4k,4\ell} \end{matrix} \quad (4) \end{aligned}$$

para cada sensor $k = 1, 2, 3, 4$, y para cada bloque de columna $\ell = 1, 2, 3, 4$. Manteniendo ℓ fijo y sumando sobre el índice del sensor k se obtiene

$$t_{4(k-1)+1,j} t_{4(k-1)+2,j} t_{4(k-1)+3,j} t_{4k,j} = \sum_{\ell=1}^4 r_{k,4(\ell-1)+1,j} r_{k,4(\ell-1)+2,j} r_{k,4(\ell-1)+3,j} r_{k,4\ell,j} \quad (5)$$

Los resultados de este enfoque no fueron tan buenos como se esperaba, ya que la lógica para controlar la secuencia de multiplicaciones y sumas involucradas en todo el proceso requería una gran cantidad de lógica dentro del FPGA.

Módulos de multiplicación de 4 columnas: El último enfoque implementado, que es la arquitectura que proporcionó los mejores resultados, se logró "equilibrando" los dos enfoques anteriores mediante la multiplicación del vector de 16 entradas x_j veces una matriz de 16×4 , es decir, multiplicando

$$\begin{aligned} & t_{4(k-1)+1,j} t_{4(k-1)+2,j} t_{4(k-1)+3,j} t_{4k,j} \\ &= x_{1,j} x_{2,j} \cdots x_{15,j} x_{16,j} \\ & \quad \begin{matrix} m_{1,4(\ell-1)+1} & m_{1,4(\ell-1)+2} & m_{1,4(\ell-1)+3} & m_{1,4\ell} \\ m_{2,4(\ell-1)+1} & m_{2,4(\ell-1)+2} & m_{2,4(\ell-1)+3} & m_{2,4\ell} \\ \vdots & \vdots & \vdots & \vdots \\ m_{15,4(\ell-1)+1} & m_{15,4(\ell-1)+2} & m_{15,4(\ell-1)+3} & m_{15,4\ell} \\ m_{16,4(\ell-1)+1} & m_{16,4(\ell-1)+2} & m_{16,4(\ell-1)+3} & m_{16,4\ell} \end{matrix} \times \begin{matrix} \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \end{matrix} \quad (6) \end{aligned}$$

Para cada bloque de columna $\ell = 1, 2, 3, 4$. Si se necesitan menos de 16 componentes principales, el índice del bloque de columna puede iterar sobre un rango menor. Cabe destacar que este enfoque tiene la ventaja de producir directamente los componentes reales de t_j , a la vez que ofrece un equilibrio razonable entre los requisitos de recursos y la cantidad de lógica requerida dentro de la FPGA. La Sección 4 presenta estadísticas sobre los recursos utilizados dentro de la FPGA. Además, este enfoque proporciona los resultados esperados y cumple con los requisitos de temporización.

3.2.4. Descripción del software

La capa superior de software se aloja en un entorno Petalinux (versión 2019.1 en este caso) proporcionado por Xilinx dentro del PS. Esto permite disponer de un sistema Linux completo, con algunas limitaciones, pero para fines de posprocesamiento, es un sistema totalmente funcional. Permite ejecutar numerosas aplicaciones y demonios. En este entorno, compilamos y ejecutamos una aplicación orientada a objetos en lenguaje C estándar, cuyos módulos/clases se muestran en la Figura 8. Después de que la sección de hardware recopile las muestras y calcule las métricas (consulte la Sección 3.2.5 para obtener más detalles), la aplicación de software almacena los datos en un objeto "DataSet" de 16 características, y luego estos datos se normalizan (cada característica se centra en su media y luego se divide por su desviación estándar) empleando un objeto "Scaler". A continuación, un objeto "PCA" calcula los vectores propios y los valores propios de la matriz de covarianza, que luego se transfieren de vuelta a la sección de hardware. La sección anterior describió la sección HW, indicando que la última acción realizada en HW fue el cálculo de la salida de PCA T veces el

que es simplemente una multiplicación del vector de métricas calculado x_j por la matriz PCA V . Esta última operación mapea el vector de métricas a un nuevo espacio ortogonalizado.

Tras esta multiplicación de alta velocidad, el vector de salida del módulo PCA se transfiere de nuevo a la capa SW superior, listo para cualquier procesamiento posterior, como la clasificación o la identificación.

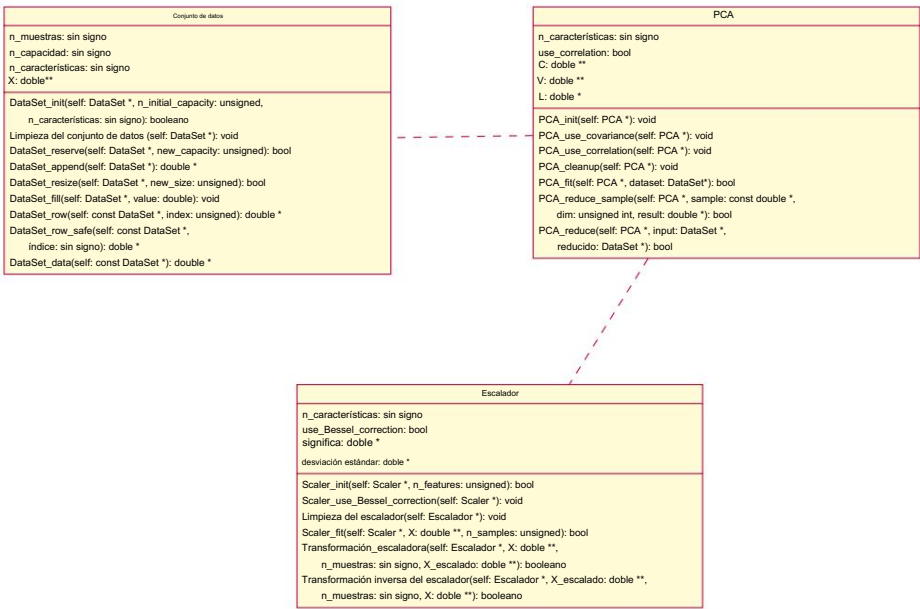


Figura 8. Diagrama de clases UML para el código en lenguaje C que calcula la matriz PCA V. Observación que * se refiere a la indirección de datos, y la doble indirección se refiere a una matriz de punteros para una dinámica matriz bidimensional.

La aplicación interactúa con los módulos implementados dentro de la sección PL de la La FPGA forma parte del sistema SoC a través de una interfaz AXI que permite enviar y recibir datos. hacia/desde los registros internos en el FPGA. El programa C obtiene datos de las “Métricas”. módulo o el módulo “PCA” a través de estos registros internos. Tanto la configuración como También se configuran diferentes modos de funcionamiento a través de esta interfaz AXI. La interfaz En la Tabla 3 se muestra la información relativa al entorno Petalinux.

El software se comunica utilizando los diferentes registros que almacenan los resultados de las operaciones. realizado dentro del área PL. Los registros se asignan a una dirección en la que todos los Los registros del sistema son accesibles. En este caso, esta dirección es 0x43C00000 (el sistema Vivado). determina esta dirección); esta dirección base se almacena en una constante, a saber, axi_ports_addr. Utilizando diferentes valores de desplazamiento, los registros que contienen las métricas son accesibles desde este dirección base (por ejemplo, el cálculo de la métrica WL para el sensor 1 está disponible en axi_ports[16], lo que significa que esta métrica son 18 direcciones de 32 bits desde la dirección base). La forma en que la capa C accede a estos registros en sus direcciones de desplazamiento se accede luego para los diferentes procesos y cálculos.

Tabla 3. Enlace de comunicación de la capa C: acceso a los registros de la FPGA en el firmware a través de la interfaz AXI.

Compensar	DIRECCIÓN	Descripción
18	43C0048	Sensor MAV 1
16	43C0040	Sensor WL 1
22	43C0058	Sensor ZC 1
30	43C0078	Sensor SSC 1
15	43C003C	Sensor MAV 2
13	43C0034	Sensor WL 2
21	43C0054	Sensor ZC 2
29	43C0074	Sensor SSC 2
12	43C0030	Sensor MAV 3
10	43C0028	Sensor WL 3
20	43C0050	Sensor ZC 3
28	43C0070	Sensor SSC 3
9	43C0024	Sensor MAV 4
7	43C001C	Sensor WL 4
19	43C004C	Sensor ZC 4
27	43C006C	Sensor SSC 4

3.2.5. Secuencia de operación y modos de operación

El sistema en la sección PL utiliza máquinas de estados finitos (FSM) para el flujo de control.

Las máquinas de estados finitos (FSM) del sistema se comunican mediante señales, utilizando la arquitectura representada en la Figura 7. El funcionamiento del sistema implica una interacción coordinada entre el Sistema de Procesamiento (PS) que ejecuta Petalinux y la Lógica Programable (PL) que contiene los módulos de procesamiento EMG. Esta interacción puede modelarse como máquinas de estados interrelacionadas que controlan el comportamiento del sistema en diferentes fases operativas.

1. Fase de inicialización:

I1 Tanto el PS como el PL inician sus respectivos componentes.

I2 El PS configura los registros PL y establece las rutas de flujo de datos.

I3 Dependiendo del modo de entrada de datos (aplicación externa basada en UART o BRAM), el PL selecciona los cuatro canales de entrada de los valores del sensor que se van a procesar.

2. Muestreo:

S1 El PS le indica al PL que recopile métricas a partir de los datos entrantes del sensor.

S2 El PL inicializa el búfer del sensor de entrada.

S3 El módulo "Procesamiento" (del PL) lee los valores del sensor de entrada (contando desde $k = 0$ hasta $N - 1$, donde N es el tamaño de la ventana) y los almacena en el búfer de entrada de 4 canales de longitud N .

S4 Una vez que hay N muestras, el módulo "Métricas" recibe una indicación del módulo "Procesamiento" para calcular inmediatamente las cuatro características del dominio del tiempo (MAV, WL, SSC, ZC) por sensor: Sensor 1

Sensor 2	Sensor 3	Sensor 4
MAV-WL-ZC-SSC	MAV-WL-ZC-SSC	MAV-WL-ZC-SSC

S5 El PL transfiere las métricas calculadas al PS, que las almacena (en un objeto DataSet) para el cálculo de la matriz PCA.

3. Modo de aprendizaje:

L1 El PS calcula la matriz de covarianza Σ a partir de los vectores de métricas recopilados.

L2 Realiza la descomposición de valores propios de Jacobi para encontrar los valores propios Λ y los valores propios vectores V .

L3 La matriz de transformación PCA V tiene sus columnas (vectores propios) ordenadas en el orden descendente adecuado según la magnitud de sus valores propios correspondientes.

L4 El PS transfiere los coeficientes de la matriz PCA V al PL a través de la interfaz AXI. El módulo "PCA" almacena estos coeficientes en registros internos.

4. Modo de operación: O1

La PS le indica a la PL que cambie al modo operativo.

O2 En lugar de transferir las métricas al PS, el módulo "PCA" ahora aplica la matriz de transformación V a los vectores de métricas, es decir, calcula $t = xTV$.
O3 Los componentes PCA resultantes t se transfieren de nuevo al PS para su posterior análisis o clasificación.

En la siguiente sección, se presentan resultados que muestran el correcto funcionamiento del sistema para un caso de ejemplo con un tamaño de ventana $N = 7$ (en la práctica, tomamos $N = 250$ o $N = 256$).

La secuencia de operaciones ilustra el correcto funcionamiento del sistema implementado. La aplicación que se ejecuta en el sistema realiza la transformación PCA de un dominio a otro para un conjunto de datos determinado, y los resultados obtenidos del sistema implementado que se ejecuta dentro de la FPGA se comparan con los del mismo conjunto de datos procesado, pero obtenidos a través de una aplicación diferente que se ejecuta en la capa Petalinux, la cual funciona como una interfaz independiente y no correlacionada.

La figura 9 proporciona una vista simplificada de la máquina de estados del sistema combinado que resalta los puntos clave de interacción entre los componentes PS y PL.

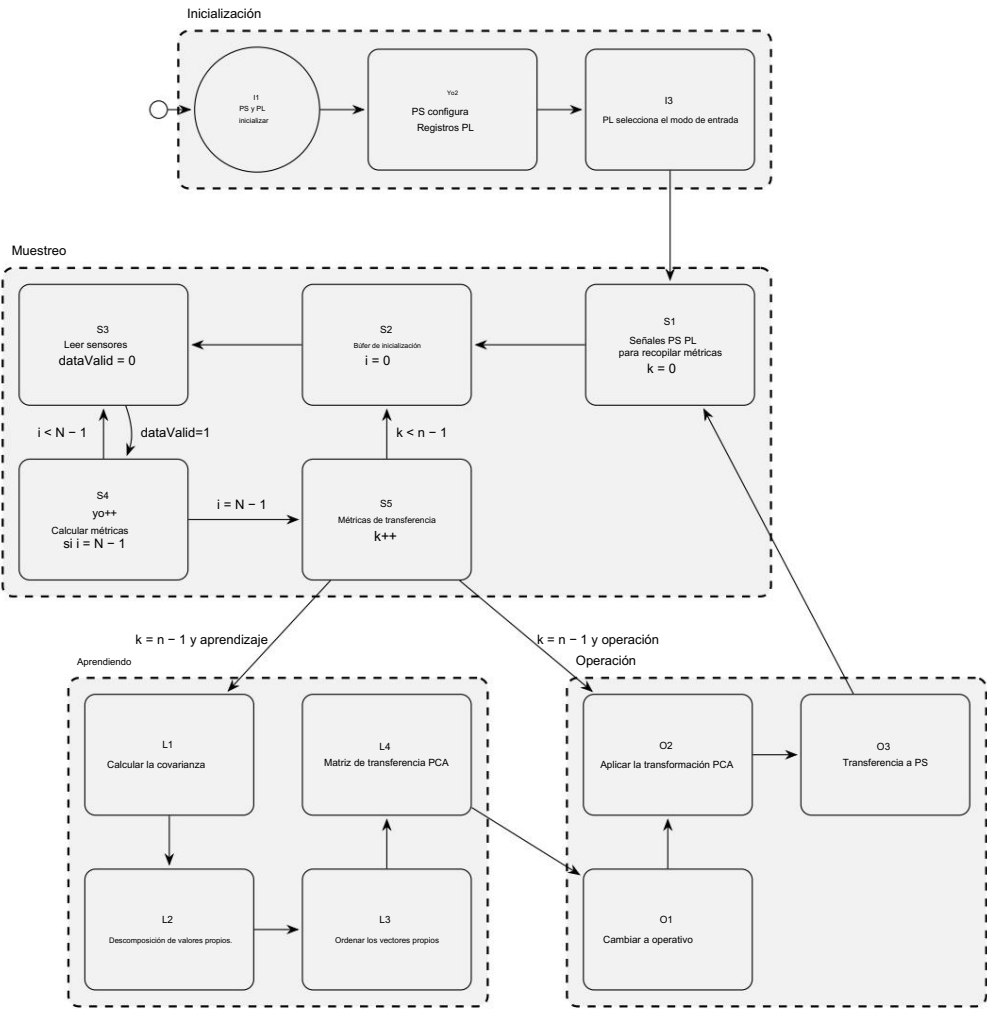


Figura 9. Máquina de estados del sistema combinado que muestra el flujo entre las operaciones PS y PL.

4. Resultados

La validación experimental se centra en las métricas de rendimiento arquitectónico en lugar de que la precisión algorítmica, ya que esta última ha sido ampliamente validada en estudios EMG previos. literatura de procesamiento. Nuestra evaluación demuestra las ganancias de eficiencia alcanzables a través de Codiseño de hardware y software en comparación con las implementaciones convencionales basadas únicamente en software.

4.1. Recursos utilizados por la FPGA

Las estadísticas sobre los recursos utilizados en el sistema FPGA-SoC se muestran en las Tablas 4 y 5, que identifican la cantidad de recursos utilizados en los diferentes módulos implementados.

Tabla 4. Informe de utilización de recursos de Vivado 2019.1. Describe cómo se utilizan los recursos disponibles. distribuidos entre los diferentes módulos.

Módulo (ID)	LUTs	Registros	F7Mux	F8Mux	Slice	LUT/Lógica	LUT/Memoria	RAM	DSPs
AXI BRAM_CTRL	184	197	0	0	86	184	0	0	0
AXI MEM_GEN	9	12	0	0	10	7	2	2	0
fp_divAav1	764	1352	0	0	257	728	36	0	0
fp_divAav2	764	1352	0	0	272	728	36	0	0

Tabla 4. Cont.

Módulo (ID)	LUTs	Registros	F7Mux	F8Mux	Slice	LUT/Lógica	LUT/Memoria	RAM	DSPs
fp_divAav3	764	1352	0	36	0	271	728	0	0
fp_divAav4	764	1352	0	0	0	315	728	36	0
fp_mulPCA0	87	166	0	0	0	79	75	12	2
fp_mulPCA1	86	166	0	0	0	71	75	11	2
fp_mulPCA2	87	166	0	0	0	76	75	12	2
fp_mulPCA3	87	166	0	0	0	79	75	11	2
mySystemip	2509	4405	64	0	0	1729	2509	0	0
módulo_de_procesamiento	24.025	19.029	2368	1184	9137	24.025	0	0	0
Sistema de procesamiento	0	0	0	0	0	0	0	0	0
AXI_Periph	537	496	0	0	213	498	39	0	0
rst_ps7_100M	16	40	0	0	15	15	1	0	0
fp_S1SSC_Div	764	1352	0	0	308	728	36	0	0
fp_S1ZC_Div	764	1352	0	0	263	728	36	0	0
fp_S2SSC_Div	764	1352	0	0	302	728	36	0	0
fp_S2ZC_Div	764	1352	0	0	279	728	36	0	0
fp_S3SSC_Div	764	1352	0	0	264	728	36	0	0
fp_S3ZC_Div	764	1352	0	0	288	728	36	0	0
fp_S4SSC_Div	764	1352	0	0	265	728	36	0	0
fp_S4ZC_Div	764	1352	0	0	286	728	36	0	0
Total	36.795	41.067	2432	1184	14.865	36.274	520	2	8

Las estadísticas de los módulos que realizan los procesos de “métricas” y “PCA” son:
También se muestra con más detalle en la Tabla 5.

Tabla 5. Utilización de recursos para los módulos más importantes del sistema: Métricas y PCA.
En esta sección, el informe se refiere a la lógica utilizada dentro de estos módulos. Las IP propietarias (para realizar La multiplicación y la división) no forman parte de esta sección del informe de utilización y los 7 multiplicadores son reportados como DSP en la Tabla 4.

Módulo (ID)	LUTs	Registros	F7Mux	F8Mux	Slice	LUT/Lógica	LUT/Memoria	RAM	DSPs
métricas del módulo	15.395	0	0	7028	0	4793	15.395	0	0
met_mod_s1	3850	1757	0	0	0	1261	3850	0	0
met_mod_s2	3851	1757	0	0	0	1214	3851	0	0
met_mod_s3	3845	1757	0	0	0	1352	3845	0	0
met_mod_s4	3849	1757	0	0	0	1236	3849	0	0
pca_module	8427	10.498	2368	1184	4731	8427	0	0	0
gen_add_mul_0	734	130	0	0	0	217	734	0	0
gen_add_mul_1	734	130	0	0	0	213	734	0	0
gen_add_mul_2	733	130	0	0	0	206	733	0	0
gen_add_mul_3	737	130	0	0	0	210	737	0	0
Total	42.155	25.074	2368	1184	15.433	42.155	0	0	0

4.2. Resultados de la aplicación

En esta sección, presentamos una secuencia del proceso realizado dentro de todo el sistema mientras procesa las muestras de entrada a través de los diferentes módulos (en evaluación modo, para que podamos comprobar el proceso en su interior).

- Una muestra de la entrada de datos para los cuatro sensores. Una muestra de los datos del sensor es
Se muestra a continuación:

-0,00436020 -0,02589910 -0,00280167 0,00802203
-0,02873340 0,00672732 0,00186870 -0,01589790
0,00176235 -0,03466060 -0,01619870 0,02321670
-0,03075060 0,03271290 0,01021340 -0,01982900
0,00216684 0,00063915 -0,04133170 -0,00858940
0,06235370 0,04674370 0,04463140 0,04991990
-0,06014440 -0,09982760 -0,05358500 -0,06827760
-0,04320210 -0,00448979 0,06629640 0,04391780
-0,02117970 0,01781130 -0,01843370 -0,03890570
-0,00947362 -0,02973350 -0,02852140 0,02709730
0,00409025 0,03296500 0,01918160 0,00211551
0,10056500 0,03024860 -0,04497290 0,03854670
-0,06442331 -0,06319550 0,00636181 -0,04414980

- Entrenamiento: Creación de la matriz PCA. Al recibir los datos, el elemento más a la izquierda representa los datos del sensor n.º 1, luego el del sensor n.º 2, y así sucesivamente. Al inicio, el sistema se encuentra en lo que denominamos "modo de aprendizaje"; durante este tiempo, los vectores de métricas se utilizan para crear la matriz PCA. En este ejemplo, el proceso se completa utilizando un tamaño de ventana N = 7, lo que significa que se requieren siete vectores de entrada de datos brutos de los sensores para obtener un vector de métricas de salida.

Muestra n.º: 1
Esperando dataValid = 1
S1:-0,004360 S2:-0,025899 S3:-0,002802 S4:0,008022

Esperando a que dataValid = 0
Muestra n.º: 2
Esperando dataValid = 1
S1:-0,028733 S2:0,006727 S3:0,001869 S4:-0,015898

Esperando datosValid = 0 Muestra#: 3
Esperando
datosValid = 1 S1:0.001762 S2:-0.034661
S3:-0.016199 S4:0.023217

Esperando a que dataValid = 0
Muestra n.º: 4
Esperando dataValid = 1
S1:-0,030751 S2:0,032713 S3:0,010213 S4:-0,019829

Esperando a que dataValid = 0
Muestra n.º: 5
Esperando dataValid = 1
S1:0,002167 S2:0,000639 S3:-0,041332 S4:-0,008589

Esperando datosValid = 0 Muestra#: 6
Esperando
datosValid = 1 S1:0.062354 S2:0.046744
S3:0.044631 S4:0.049920

Esperando a que dataValid = 0
Muestra n.º: 7
Esperando dataValid = 1
S1:-0,060144 S2:-0,099828 S3:-0,053585 S4:-0,068278

Esperando a que dataValid = 0
¡Métricas completadas!

Entrada al módulo PCA:
S1:AAV-WL-ZS-SSC; S2:AAV-WL-ZS-SSC;
0,027182 0,302984 4,000000 4,000000

```
0,035316 0,366137 4,000000 5,000000
S3:AAV-WL-ZS-SSC; S4:AAV-WL-ZS-SSC;
0,024376 0,284874 6,000000 5,000000
0,027679 0,294027 5,000000 4,000000
Ejemplo para el cálculo de PCA
```

Una vez alcanzado el número indicado de vectores de métricas, el sistema entra en "modo operativo". En este ejemplo, el número requerido de vectores de métricas de entrada es $M = 7$. Por lo tanto, en nuestro caso, necesitamos siete (7) ventanas de tamaño $N = 7$, así que en total, necesitamos procesar $NM = 7 \times 7 = 49$ muestras.

```
Muestra n.º: 43
Esperando dataValid = 1
S1:-0,041569 S2:0,057423 S3:0,025218 S4:-0,018170

Esperando datosValid = 0 Muestra#: 44
Esperando
datosValid = 1 S1:0.033394 S2:-0.079262
S3:-0.038959 S4:0.036694

Esperando a que dataValid = 0
Muestra n.º: 45
Esperando dataValid = 1
S1:0,006929 S2:0,051741 S3:-0,001391 S4:0,020102

Esperando a que dataValid = 0
Muestra n.º: 46
Esperando dataValid = 1
S1:0,026344 S2:0,031666 S3:0,025776 S4:0,027777

Esperando a que dataValid = 0
Muestra n.º: 47
Esperando dataValid = 1
S1:-0,036288 S2:-0,031773 S3:-0,018986 S4:-0,104803

Esperando datosValid = 0 Muestra#: 48
Esperando
datosValid = 1 S1:-0.070014 S2:0.032346
S3:0.067164 S4:0.001208

Esperando a que dataValid = 0
Muestra n.º: 49
Esperando dataValid = 1
S1:0,067987 S2:0,035339 S3:-0,005998 S4:0,092489

Esperando a que dataValid = 0
¡Métricas completadas!

Entrada al módulo PCA:
S1:AAV-WL-ZS-SSC; S2:AAV-WL-ZS-SSC;
0,040361 0,355202 3,000000 4,000000
0,045650 0,418315 4,000000 3,000000
S3:AAV-WL-ZS-SSC; S4:AAV-WL-ZS-SSC;
0,026213 0,332987 5,000000 4,000000
0,043035 0,409004 3,000000 4,000000
Ejemplo para el cálculo de PCA
```

Los procesos completados después de capturar los datos de entrada sin procesar para la muestra n.º 49 son los siguientes: – Calcular la covarianza utilizando los datos recibidos (los datos de los vectores de métricas, valores calculados derivados de los datos del sensor de entrada sin procesar).

- Calcular el coeficiente jacobiano para la matriz creada.
 - Calcular el sistema de autovalores: autovalores y autovectores.
 - Determinar los coeficientes de la matriz PCA.
 - Transferir los coeficientes de la matriz PCA a los registros internos de la FPGA para que sean utilizados por el módulo PCA en el FW.
- Modo de operación. Una vez que el sistema está en funcionamiento y la secuencia es la misma, esperamos (N = 7 en el ejemplo mostrado) vectores de datos de entrada sin procesar para obtener un vector de métricas de salida, que luego es procesado por el módulo PCA. La salida de la implementación FPGA se compara con otro proceso que se ejecuta en el sistema Petalinux para fines comparativos, y se muestra como "Componentes PCA calculados".

El módulo PCA integrado en la FPGA genera un vector de resultados que representa los componentes PCA. A continuación se muestra un ejemplo de dicho vector.

Resultados del FPGA:

```
PCA Comp 00_03 3.474750 6.473070 -1.006910 1.853277 PCA Comp 04_07 2.989338
3.810144 0.082271 -2.816533 PCA Comp 08_11 0.116517 -0.031517 1.053864
-0.191054 PCA Comp 12_15 -0.008754 1.873763 0.036685~-0.136512
```

Componentes PCA calculados: PCA

```
Comp 00_03 3.474750 6.473071 -1.006910 1.853276 PCA Comp 04_07 2.989338
3.810143 0.082271 -2.816533 PCA Comp 08_11 0.116517 -0.031517 1.053864
-0.191054 PCA Comp 12_15 -0.008754 1.873763 0.036685 -0.136512
```

La Figura 10 muestra una captura de analizador lógico que mide la duración de cada subproceso durante el último ciclo de procesamiento de entrada de datos del sensor. La sección de 1,8 μ s representa el intervalo del proceso de cálculo para las métricas. El pulso en la línea de tiempo D0 sirve como señal de inicio para el i-ésimo paso en la secuencia de métricas. Una vez que comienza el proceso, las diferentes métricas se calculan en paralelo. La línea de tiempo D1 es la duración del tiempo para las métricas MAV y WL (se procesan simultáneamente); la línea de tiempo D2 es la duración para la métrica ZC y la línea de tiempo D3 es la duración para la métrica SSC. El tiempo de proceso para cada métrica es diferente; por lo tanto, un módulo monitorea e indica cuándo se completan todas las métricas y proporciona un pulso que representa la duración total del tiempo para todos los cálculos de métricas; este pulso se muestra en la línea de tiempo D4. De esta manera, se realizan N – 1 procesos de métricas; Luego, una vez completado el último proceso, comienza el proceso de multiplicación (que forma parte de la tarea realizada por el módulo PCA), el cual tarda 13,8 μ s. La figura 10 muestra la duración de cada uno de estos procesos.

Para algunas aplicaciones de EMG, el tiempo de muestreo es del orden de 1 ms, por lo que completar este proceso en 15,6 μ s permite utilizar el tiempo restante para otros procesos, como la clasificación. Es importante mencionar que este artículo no trata sobre clasificación (ya que se describirá en una publicación posterior). Por lo tanto, para un proceso con N = 256, el tiempo para procesar una secuencia completa sería de 250 ms, y solo en la última muestra de entrada el proceso tarda más (en este caso, 15,6 μ s), lo que permite, como se indicó anteriormente, disponer prácticamente de todo el período de muestreo para otros procesos y/o cálculos.

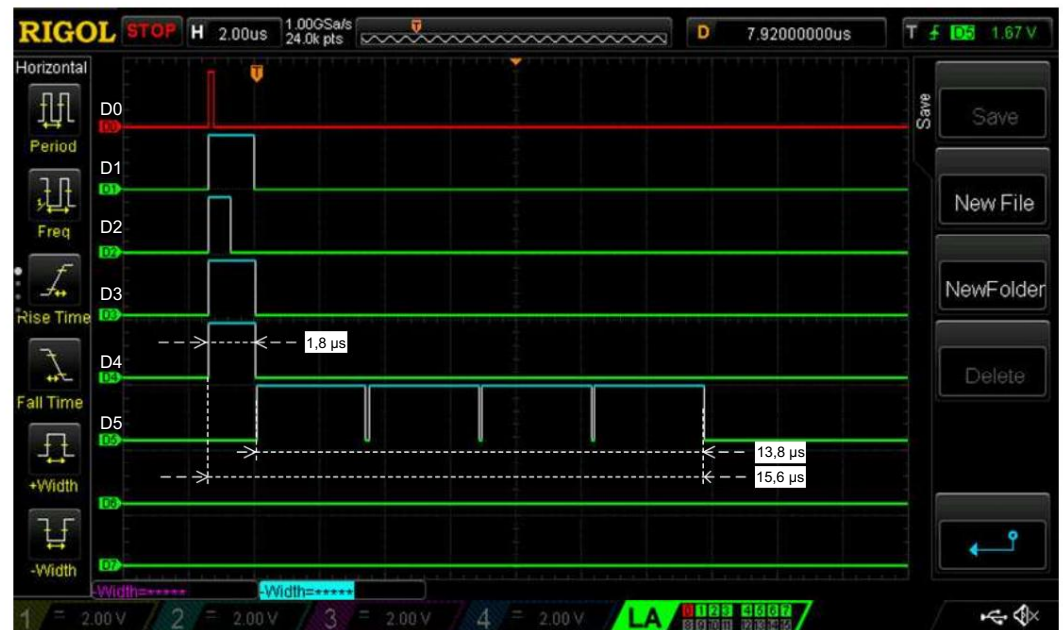


Figura 10. Medición de los intervalos de tiempo mediante osciloscopio: D0: DataValid; Métricas: D1: MAV y WL disponibles; D2: ZC disponible; D3: SSC disponible; D4: Tiempo disponible de todas las métricas (OR D1, D2 y D3); D5: Procesos de multiplicación de PCA.

5. Conclusiones y trabajo futuro

Este trabajo presenta un estudio de caso arquitectónico que demuestra la viabilidad y las ventajas de integrar la extracción de características EMG con la reducción de dimensionalidad basada en PCA en sistemas FPGA embebidos. El hallazgo clave es que las arquitecturas de coprocesamiento pueden lograr mejoras significativas en el rendimiento (7,3 veces más velocidad, 3,1 veces más eficiencia energética) en comparación con los enfoques de software tradicionales, lo que las hace idóneas para aplicaciones protésicas alimentadas por batería. La metodología arquitectónica presentada aquí puede extenderse a otros algoritmos de extracción de características y técnicas de reducción de dimensionalidad, proporcionando una base para futuros sistemas de procesamiento EMG embebidos.

Este trabajo consiste en un sistema modular que implementa algunas de las métricas más comunes utilizadas en el campo de la electromiografía (EMG). Estas métricas se implementaron en VHDL, y su interfaz está integrada en un sistema Petalinux que interactúa con el firmware ubicado en el área de lógica programable. El objetivo de este trabajo es obtener un sistema con potencial para su uso en una aplicación portátil para prótesis transtibiales.

Con el propósito de comparar los recursos FPGA reportados en este artículo, los autores elaboraron la Tabla 6, donde se comparan dos sistemas similares, el primero para la eliminación de ruido de ECG [38] y el segundo para bloques de preprocesamiento [39].

Debido a las limitaciones de recursos de la placa de evaluación, algunos bloques de filtros se implementan actualmente fuera del FPGA. Sin embargo, si se utiliza un FPGA de mayor capacidad, el banco de filtros para limpiar la señal de los sensores se puede añadir al firmware, lo que permite un sistema más robusto.

Otras posibles áreas futuras consisten en añadir un sistema RTOS dentro del sistema Petalinux para que el entrenamiento de la matriz PCA se pueda realizar como un proceso periódico, lo que permite que el sistema aprenda del historial de las muestras que entran en el sistema y que se procesan a través de él.

Este trabajo se centra en las contribuciones arquitectónicas más que en la innovación algorítmica. La selección de características del dominio del tiempo y PCA representa métodos establecidos elegidos específicamente por sus características de implementación de hardware. La principal contribución es la demostración de que las arquitecturas de hardware integradas pueden mejorar significativamente el rendimiento.

mejorar la eficiencia en sistemas EMG integrados con recursos limitados, proporcionando una base para futuras aplicaciones de control de prótesis.

Tabla 6. Comparación entre el presente trabajo y dos aplicaciones similares.

Proyecto	Secciones		Rebanada DUT	Potencia (W)
Artículo sobre la eliminación de ruido en el ECG [38]	Ondícula			
	Implementaciones			
	Ondícula Haar 18		309	0,262
	Ondícula Daubechies 34		729	0,316
	Coiflets de ondículas 34		729	0,313
Totales	Ondícula biorthogonal 34 120		729	0,301
			2496	1.192
Preprocesamiento del documento [39]				
	Hanning	7	225	0,144
	Emperador	6	220	0,121
	Hamming	6	220	0,136
	Bartlett	6	220	0,121
	Rectangular	6	247	0,136
	Hombre negro	7	247	0,142
	Totales	38	1379	0,8
Este trabajo	Módulos y Principal			
	Áreas			
	Módulo de métricas	0	15.412	
	Módulo PCA	8	10.507	
	Relojes			0,104
	Señales			0,179
	Lógica			0,191
	BRAM y E/S			0,006
Totales		8	25.919	0,009
				0,489

Contribuciones de los autores: Conceptualización, CGM-P. y RAG-L.; Curación de datos, MA; Formal
Análisis, JR-R.; Investigación, DCT-P.; Metodología, DCT-P.; Administración del proyecto, CGM-P.
y RAG-L.; Recursos, DCT-P.; Software, CGM-P. y RAG-L.; Supervisión, JR-R.; Validación,
CGM-P.; Redacción: borrador original, CGM-P. y RAG-L.; Redacción: revisión y edición, MA All

Los autores han leído y aprobado la versión publicada del manuscrito.

Financiación: Los autores desean agradecer a SECIHTI de México por la financiación necesaria para generar las bases de datos utilizadas para esta investigación.

Declaración de consentimiento informado: En este trabajo se utilizó una base de datos de una investigación previa. La base de datos se obtuvo con la colaboración de ocho sujetos; sin embargo, este trabajo solo utiliza la base de datos y no hubo interacción con estas personas.

Declaración de disponibilidad de datos: La base de datos está disponible a solicitud de los autores.

Conflictos de intereses: Los autores declaran no tener conflictos de intereses.

Abreviaturas

En este manuscrito se utilizan las siguientes abreviaturas:

Interfaz extensible avanzada AXI

Electrocardiogramas (ECG)

Electromiografía (EMG)

Matriz de puertas programables en campo (FPGA)

Máquina de estados finitos (FSM)

Firmware FW

Hardware HW

SI Interfaz

IP Propiedad intelectual

Valor absoluto medio (VAM)

Análisis de componentes principales (PCA)
 PL Lógica programable
 PD Sistema de procesamiento
 Nivel de transferencia de registros RTL
 SSC Cambios en la señal de pendiente
 Software de software
 UART (Transmisor-Receptor Asíncrono Universal)
 Longitud de la forma de onda WL
 ZC Cruces por cero

Apéndice A

A continuación se presenta un resumen de las características comunes extraídas de las señales EMG en diferentes dominios.

Características en el dominio del tiempo: Estas características se calculan directamente a partir de la señal EMG sin procesar.

- Valor absoluto medio (VAM):

$$\text{MAV} = \frac{1}{N} \sum_{i=1}^{\text{norte}} |x[i]|.$$

Esto también se conoce como Valor Rectificado Promedio (VRP) y es una característica comúnmente utilizada para cuantificar la actividad muscular y puede utilizarse para evaluar la fatiga muscular. • Raíz Cuadrada Media (RMS):

$$\text{RMS} = \sqrt{\frac{1}{\text{norte}} \sum_{i=1}^{\text{norte}} (x[i])^2}.$$

El valor RMS es proporcional a la amplitud de la señal EMG y refleja la intensidad de la contracción muscular. El RMS es menos sensible al ruido que el MAV.

- Cruces por cero (ZC): Cuenta el número de veces que la señal cruza por cero con un umbral para evitar el ruido.
- Cambios de signo de pendiente (SSC): Cuenta el número de cambios en la dirección de la pendiente, lo que indica fluctuaciones en la señal. •

Longitud de forma de onda (WL):

$$\text{WL} = \sum_{i=1}^{N-1} |x[i+1] - x[i]|$$

- Electromiografía integrada (EMG):

$$\text{IEMG} = \sum_{i=1}^{\text{norte}} |x[i]|$$

Características en el dominio de la frecuencia: Estas características se obtienen después de aplicar una transformación como la Transformada Rápida de Fourier (FFT).

- Frecuencia media (FMN):

$$\text{MNF} = \frac{\sum f_k \cdot P_k}{P_k}$$

- Frecuencia mediana (MDF): La frecuencia que divide el espectro de potencia en dos mitades iguales.
- Entropía del espectro de potencia (PSE): Mide la distribución de potencia en diferentes frecuencias.
- Potencia total: La suma de todas las densidades espectrales de potencia, que representa la potencia total de la señal.

Características del dominio tiempo-frecuencia: Estas características proporcionan información sobre la señal tanto en el dominio del tiempo como en el de la frecuencia. • Coeficientes de ondículas: Extraídos mediante descomposición de ondículas, capturando tanto el tiempo y la información de frecuencia.

• Características del espectrograma: Contenido de frecuencia variable en el tiempo, útil para señales no estacionarias.

Características estadísticas: Estas características describen las propiedades estadísticas de la señal EMG. • Media: El valor promedio de la señal. • Desviación estándar (DE): Mide la variabilidad de la señal. • Asimetría: Indica la asimetría de la distribución de la señal. • Curtosis: Mide la asimetría de la distribución de la señal.

Características no lineales: Estas características capturan la naturaleza compleja y no lineal de las señales EMG. • Entropía aproximada (ApEn): Mide la complejidad de la señal, donde valores más altos indican mayor irregularidad.

• Dimensión fractal (DF): Describe la autosimilitud de la señal. • Exponente de Lyapunov: Mide el comportamiento caótico de la señal; valores positivos indican caos.

Referencias

- Namboothiripad, MK; Vadhyan, G. Implementación eficiente de redes neuronales artificiales en FPGAs mediante síntesis de alto nivel y paralelismo. *Int. J. Adv. Technol. Eng. Explor.* 2024, 11, 1497–1511. [\[CrossRef\]](#)
- Fejér, A.; Nagy, Z.; Benois-Pineau, J.; Szolgay, P.; de Rugy, A.; Domenger, JP Implementación de un detector de transformada de características invariante a escala en FPGA para dispositivos portátiles de bajo consumo para el control de prótesis. *Int. J. Circuit Theory Appl.* 2021, 49, 2255–2273. [\[CrossRef\]](#)
- Pal, S.; Upadhyaya, BK; Majumder, T.; Dasgupta, S.; Das, N.; Bhattacharjee, A. Optimización de la configuración dinámica de aceleradores FPGA mediante aprendizaje por refuerzo para mejorar el rendimiento y la utilización de recursos. *Eng. Res. Express* 2025, 7, 015317. [\[CrossRef\]](#)
- Patel, V.; Shah, A. Diseño e implementación de un filtro multibanda óptimo de bajo consumo basado en FPGA con función Spline para la reducción de ruido en señales de ECG. *Comput. Methods Biomech. Biomed. Eng.* 2025, 28, 226–237. [\[CrossRef\]](#)
- Reddy, VHP; Kumar, PK Reconstrucción de señal ECG habilitada por FPGA basada en una búsqueda de coincidencia ortogonal mejorada Algoritmo. *Integración* 2025, 101, 102311. [\[CrossRef\]](#)
- Cai, Z.; Li, P.; Cheng, L.; Yuan, D.; Li, M.; Li, H. Arquitectura de hardware heterogénea de alto rendimiento para interfaz cerebro-computadora. *Biomed. Eng. Lett.* 2024, 15, 217–227. [\[CrossRef\]](#)
- Pillai, HH; PS, LP; Ekanayaka, KU; Suthakorn, J.; Pillai, BM. Sistema basado en FPGA activado por bioseñales para rehabilitación robótica. En *Actas de la 3.ª Conferencia Internacional sobre Robótica, Automatización e Inteligencia Artificial (RAAI) de 2023*, Singapur, 14-16 de diciembre de 2023; págs. 195-199. [\[CrossRef\]](#)
- Perry, J.; Bekey, G. Relaciones entre la electromiografía y la fuerza en el músculo esquelético. *Crit. Rev. Biomed. Eng.* 1981, 7, 1–22. [\[PubMed\]](#)
- Gao, B.; Han, Y.; Zhou, Y.; Yu, J.; Li, S.; Dong, A. Sistema inalámbrico portátil de adquisición de sEMG multicanal con electrodos secos. En *Actas de la conferencia Wireless Artificial Intelligent Computing Systems and Applications*; Cai, Z., Takabi, D., Guo, S., Zou, Y., Eds.; Springer: Cham, Suiza, 2025; pp. 124–135. [\[CrossRef\]](#)
- Gautam, Y.; Jebelli, H. Diseño de un sensor EMG serpentino flexible basado en poliimida para la detección de fatiga mediante IA en la construcción. *Sens. Bio-Sens. Res.* 2024, 46, 100713. [\[CrossRef\]](#)
- Hambly, MJ; de Sousa, ACC; Pizzolato, C. Comparación de métodos de filtrado para la extracción en tiempo real de la EMG voluntaria. *Componente en músculos estimulados eléctricamente. Biomed. Signal Process. Control* 2024, 87, 105471. [\[CrossRef\]](#)
- Esposito, D.; Centracchio, J.; Bifulco, P.; Andreozzi, E. Un enfoque inteligente para la extracción de la envolvente EMG y una potente eliminación de ruido para Interfaces hombre-máquina. *Sci. Rep.* 2023, 13, 7768. [\[CrossRef\]](#)
- Choi, HS Clasificación de señales de electromiograma (EMG) basada en redes neuronales ligeras con FPGA para dispositivos portátiles Aplicación. *Electrónica.* 2023, 12, 1398. [\[CrossRef\]](#)
- Kok, CL; Ho, CK; Tan, FK; Koh, YY Extracción de características y clasificación de señales EMG basadas en aprendizaje automático para Control protésico intuitivo. *Appl. Sci.* 2024, 14, 5784. [\[CrossRef\]](#)
- Dhanush Babu, R.; Siva Adithya, S.; Dhanalakshmi, M. Diseño y desarrollo de una prótesis transfemoral controlada por EMG. *Meas. Sensors* 2024, 36, 101399. [\[CrossRef\]](#)

16. Boschmann, A.; Agné, A.; Thombansen, G.; Witschen, L.; Kraus, F.; Platzner, M. Aceleración basada en Zynq de alta densidad robusta Procesamiento de señales mioeléctricas. *J. Parallel Distrib. Comput.* **2019**, *123*, 77–89. [\[CrossRef\]](#)
17. Farina, D.; Merletti, R.; Enoka, RM La extracción de estrategias neuronales a partir de la EMG de superficie: 2004–2024. *J. Appl. Physiol.* **2025**, *138*, 121–135. [\[CrossRef\]](#)
18. Andalib, A.; Farina, D.; Vujaklija, I.; Negro, F.; Aszmann, OC; Bashirullah, R.; Principe, JC Decodificación no supervisada de trenes de impulsos de motoneuronas espinales para estimar la cinemática de la mano tras la reinervación muscular dirigida. *arXiv* **2019**. [\[CrossRef\]](#)
19. Konuk, ME; Sahin, DO; Kılıç, E. EMG Verilerin Sınıflandırılmasında PCA Boyut İndirgeme Tekniğinin Etkisinin İncelenmesi/Investigando el efecto de la técnica de reducción de dimensiones PCA en la clasificación de datos EMG. En *Actas del Congreso Internacional de 2024 sobre Interacción, Optimización y Aplicaciones Robóticas entre Personas y Computadoras (HORA)*, Estambul, Türkiye, 23 a 25 de mayo de 2024; págs. 1–6. [\[Referencia cruzada\]](#)
20. Bosco, G. Análisis de componentes principales de señales electromiográficas: una visión general. *Open Rehabil. J.* **2010**, *3*, 127–131. [\[CrossRef\]](#)
21. Zhu, M.; Guan, X.; Li, Z.; He, L.; Wang, Z.; Cai, K. Predicción del movimiento de las extremidades inferiores basada en sEMG utilizando CNN-LSTM con mejora Algoritmo de optimización PCA. *J. Bionic Eng.* **2023**, *20*, 612–627. [\[CrossRef\]](#)
22. Matrone, GC; Cipriani, C.; Carrozza, MC; Magenes, G. Control mioeléctrico en tiempo real de una prótesis de mano multifinger usando Análisis de componentes principales. *J. NeuroEng. Rehabil.* **2012**, *9*, 40. [\[CrossRef\]](#)
23. Cabegin, K.; Lim, M.; Fernan, DT; Garcia Santos, R.; Magwili, G. Control de brazo protésico basado en electromiografía para amputados transradiales mediante análisis de componentes principales y algoritmos de máquinas de vectores de soporte. En *Actas de la 11.ª Conferencia Internacional IEEE de 2019 sobre Humanoides, Nanotecnología, Tecnologías de la Información, Comunicación y Control, Medio Ambiente y Gestión, HNICEM 2019*, Laoag, Filipinas, 29 de noviembre - 1 de diciembre de 2019. [\[CrossRef\]](#)
24. Geethanjali, P. Estudio comparativo de PCA en la clasificación de señales EMG multicanal. *Australas. Phys. Eng. Sci. Med.* **2015**, *38*, 331–343. [\[CrossRef\]](#) [\[PubMed\]](#)
25. Qi, J.; Jiang, G.; Li, G.; Sun, Y.; Tao, B. Sistema de reconocimiento de gestos de la mano mediante EMG de superficie basado en PCA y GRNN. *Neural Comput. Solicitud* **2020**, *32*, 6343–6351. [\[CrossRef\]](#)
26. Bachanna, P.; Gadgay, B.; Chatterjee, S. Extracción de características basada en análisis de componentes principales probabilístico de aplicaciones de sistemas embebidos con implementación basada en redes neuronales profundas en FPGA. *Int. J. Recent Innov. Trends Comput. Commun.* **2023**, *11*, 45–51. [\[CrossRef\]](#)
27. Yadav, M.; Koul, R.; Suneja, K. Diseño de hardware basado en FPGA de PCA para reconocimiento facial. En *Actas de la 7.ª Conferencia Internacional sobre Procesamiento de Señales y Redes Integradas (SPIN) de 2020*, Noida, India, 27–28 de febrero de 2020; págs. 642–646. [\[CrossRef\]](#)
28. Ghasemzadeh, H.; Fallahzadeh, R.; Jafari, R. Un modelo de procesamiento energéticamente eficiente asistido por hardware para el reconocimiento de actividad mediante dispositivos portátiles. *ACM Trans. Des. Autom. Electron. Syst.* **2016**, *21*, 1–27. [\[CrossRef\]](#)
29. Castruita-López, JF; Avilés, M.; Toledo-Pérez, DC; Macías-Socarrás, I.; Rodríguez-Reséndiz, J. Señales de electromiografía en sistemas empotrados: una revisión de técnicas de procesamiento y clasificación. *Biomimética* **2025**, *10*, 166. [\[CrossRef\]](#) [\[PubMed\]](#)
30. Cerina, L.; Cancian, P.; Franco, G.; Santambrogio, MD Aceleración por hardware para la factorización de matrices no negativas de EMG de superficie. En *Actas de los Talleres del Simposio Internacional de Procesamiento Paralelo y Distribuido de la IEEE de 2017 (IPDPSW)*, Lake Buena Vista, FL, EE. UU., 29 de mayo–2 de junio de 2017; págs. 168–174. [\[CrossRef\]](#)
31. Najafi, TA; Calero, JAM; Thévenot, J.; Duc, B.; Albin, S.; Amirshahi, A.; Tají, H.; Beneyto, MJB; Affanni, A.; Atienza, D. VersaSens: una plataforma multimodal extensible para dispositivos portátiles de IA de borde de próxima generación. *IEEE Trans. Circuits Syst. Artif. Intell.* **2024**, *1*, 83–96. [\[CrossRef\]](#)
32. Meddah, K.; Zairi, H.; Bessekri, B.; Cherrih, H.; Kadir-Talha, M. Implementación en FPGA de la detección de crisis epilépticas basada en DWT, PCA y máquina de vectores de soporte. En *Actas de la Segunda Conferencia Internacional sobre Sistemas Embebidos y Distribuidos (EDIS) de 2020*, Orán, Argelia, 3 de noviembre de 2020; págs. 141–146. [\[CrossRef\]](#)
33. Cerina, L.; Franco, G.; Cancian, P.; Santambrogio, MD. Robustez de clasificadores de EMG de superficie con descomposición de punto fijo en arquitectura reconfigurable. En *Actas de los Talleres del Simposio Internacional de Procesamiento Paralelo y Distribuido de la IEEE de 2018 (IPDPSW)*, Vancouver, BC, Canadá, 21–25 de mayo de 2018; págs. 146–153. [\[CrossRef\]](#)
34. Lv, H.; Wang, Y.; Hao, B. Estimación del ángulo articular de las extremidades inferiores basada en señales de electromiografía de superficie. *Biomed. Signal Process. Control* **2025**, *104*, 107563. [\[CrossRef\]](#)
35. Kokkinis, A.; Siozios, K. Estimación rápida de recursos de aceleradores MLP basados en FPGA para aplicaciones TinyML. *Electronics* **2025**, *14*, 247. [\[CrossRef\]](#)
36. Press, WH; Teukolsky, SA; Vetterling, WT; Flannery, BP *Recetas numéricas: Libro de ejemplos C*, 2.ª ed.; Cambridge University Press: Cambridge, Reino Unido, 1992.
37. Fay, R.; Hsieh, A.; Jeang, D.; Jenkins, B. Biblioteca de punto flotante. **2023**. Disponible en línea: [https://github.com/xesscorp/Floating_Biblioteca Point-JHU](https://github.com/xesscorp/Floating_Biblioteca_Point-JHU) (Consultado el 5 de mayo de 2023).

-
38. Kirti; Sohal, H.; Jain, S. Implementación en FPGA de módulos de preprocesamiento de secuencias y colaterales para la eliminación de ruido de ECG de bajo consumo. Módulo. Inform. Med. Unlocked 2022, 28, 100838. [\[CrossRef\]](#)
39. Sohal, H.; Jain, S. Implementación en FPGA de un bloque de preprocesamiento de ECG de bajo consumo energético; Universidad Jaypee de Tecnología de la Información: Solan, India, 2019.

Aviso legal/Nota del editor: Las declaraciones, opiniones y datos contenidos en todas las publicaciones son responsabilidad exclusiva de sus autores y colaboradores, y no de MDPI ni de sus editores. MDPI y sus editores no se responsabilizan de los daños a personas o bienes que pudieran derivarse de las ideas, métodos, instrucciones o productos mencionados en el contenido.